

Lecture 13

Tuesday March 10th, 2026

Important reminders

- Start reading the pintos documentation
 - getting started, threads, synchronization
- First part is implementing an alarm clock
- We are only doing project 1

Synchronization Hardware

- Many systems provide hardware support for critical section code
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - **Atomic** = non-interruptable
 - Either test memory word and set value
 - Or swap contents of two memory words

Semaphore

- `wait()` or `P()`
- `signal()` or `V()`
- functions to organize thread access to variables or Critical Regions

Example

- Watch recording for class example with three threads
- T1, T2, T3 all want to increment the global variable count
- Need a lock for the variable
- First thread T1 locks the count variable and T2 and T3 must wait
- When T1 is done it will unlock and send the signal that the variable is free
- code counter.c is included on lecture 13 page in ilearn

Semaphores as Synchronization Tool

- **Counting semaphore** - integer value can range over an unrestricted domain
- **Binary semaphore** - integer value can range only between 0 and 1; can be simpler to implement
 - also known as **mutex locks**
- provides mutual exclusion

Classical Problems of Synchronization

- Bounded-Buffer Problem
- Readers and Writers Problem
- Dining-Philosophers Problem
- These are all **categories** to problems

Bounded Buffer Problem

- Shared buffer with N slots to store at most N items
- Producer processes data items and puts them into the buffer
- Consumer gets the data items from the buffer
- Variable empty keeps number of empty slots in the buffer
- Variable full keeps number of full items in the buffer

Bounded Buffer Semaphore Solution

- **Semaphore mutex** for access to the buffer, initialized to 1
- **Semaphore full** (number of full buffers) initialized to 0
- **Semaphore empty** (number of empty buffers) initialized to N

Readers Writers Problem

- Multiple Readers and Writers concurrently accessing the same database
- Multiple Readers accessing at the same time is OK
- When a Writer is accessing, there should be no other process accessing at the same time